

Rezolvare Completă și Detaliată

Examenul Național pentru Definitivare în Învățământ

Anul 2019 - Informatică și Tehnologia Informației

Cuprins

I. Rezolvare Definitivat Model 2019	2
SUBIECTUL I (60 de puncte)	2
1. Algoritmul de căutare binară	2
2. Structura de bază a unei pagini web	2
3. Programare: Căutare subșir în șir circular	2
4. Baze de date: Cabinet Medical	3
II. Rezolvare Definitivat Varianta 3 (24 iulie 2019)	3
SUBIECTUL I	3
1. Algoritmul de parcurgere în adâncime (DFS)	3
2. Structura sistemelor de calcul: Tastatura	4
3. Programare: Cifrele minime și maxime distincte	4
4. Baze de date: Agenție de impresariat muzical	5

I. Rezolvare Definitivat Model 2019

SUBIECTUL I (60 de puncte)

1. Algoritmul de căutare binară

- **Descriere:** Căutarea binară se aplică exclusiv pe vectori ordonați. Algoritmul compară valoarea căutată cu elementul din mijlocul vectorului. Dacă sunt egale, elementul a fost găsit. Dacă valoarea căutată este mai mică, căutarea continuă în jumătatea stângă, altfel în jumătatea dreaptă. Procesul se repetă până când elementul este găsit sau intervalul devine vid.
- **Trace pentru 7 elemente ordonate:** [2, 5, 8, 12, 16, 23, 38], căutăm valoarea 16:
 1. $st = 0, dr = 6, mij = 3$. $V[3] = 12 < 16$. Continuăm la dreapta.
 2. $st = 4, dr = 6, mij = 5$. $V[5] = 23 > 16$. Continuăm la stânga.
 3. $st = 4, dr = 4, mij = 4$. $V[4] = 16 == 16$. Elementul a fost găsit pe poziția 4.
- **Complexitate:** $O(\log N)$ în cel mai rău și mediu caz, $O(1)$ în cel mai bun caz.
- **Problemă (Căutare Binară):**

```
#include <iostream>
using namespace std;
int cautareBinara(int A[], int n, int x) {
    int st = 0, dr = n - 1;
    while (st <= dr) {
        int m = st + (dr - st) / 2;
        if (A[m] == x) return m;
        if (A[m] < x) st = m + 1;
        else dr = m - 1;
    }
    return -1;
}
int main() {
    int A[] = {2, 5, 8, 12, 16, 23, 38};
    cout << cautareBinara(A, 7, 16) << endl;
    return 0;
}
```

2. Structura de bază a unei pagini web

- **HTML:** Limbaj de marcare descriptiv ce folosește etichete (tags).
- **Structură:** <html>, <head> (metadate, titlu), <body> (conținutul propriu-zis).
- **Organizare conținut:** Etichete pentru paragrafe (<p>), titluri (<h1>-<h6>), liste (,), legături (<a>), imagini ().

3. Programare: Căutare subșir în șir circular

Soluție C++:

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
```

```

ifstream fin("def2019.in");
string s, t;
if (!(fin >> s >> t)) return 0;
fin.close();

// Verificăm dacă t este subșir în s duplicat (s + s)
string dublat = s + s;
ofstream fout("def2019.out");
if (dublat.find(t) != string::npos) {
    fout << "DA\n";
} else {
    fout << "NU\n";
}
fout.close();
return 0;
}

```

4. Baze de date: Cabinet Medical

• Entități:

- PACIENT: id_pacient (PK), nume, prenume, data_nasterii.
- MEDIC: id_medic (PK), nume, prenume, specializare.
- CONSULTATIE: id_consultatie (PK), id_pacient (FK), id_medic (FK), data_ora, diagnostic.

II. Rezolvare Definitivat Varianta 3 (24 iulie 2019)

[Subiect PDF](file:

SUBIECTUL I

1. Algoritmul de parcurgere în adâncime (DFS)

- **Descriere:** DFS utilizează o stivă (sau recursivitate). Pornește dintr-un nod sursă, îl marchează ca vizitat, apoi vizitează recursiv primul vecin nevizitat al acestuia. Când un nod nu mai are vecini nevizitați, se întoarce (backtrack) la nodul anterior pentru a continua căutarea.
- **Trace pe graf cu 10 noduri:** Exemplificarea marcării nodurilor vizitate.
- **Complexitate:** $O(V + E)$ (noduri + muchii) pentru liste de adiacență, $O(V^2)$ pentru matrice de adiacență.
- **Problemă (DFS):**

```

#include <iostream>
#include <vector>
using namespace std;
void dfs(int nod, const vector<vector<int>> &adj, vector<bool> &viz) {
    viz[nod] = true;
    cout << nod << " ";
    for (int vecin : adj[nod]) {
        if (!viz[vecin]) dfs(vecin, adj, viz);
    }
}

```

2. Structura sistemelor de calcul: Tastatura

- **Integrare:** Dispozitiv periferic de intrare ce comunică cu placa de bază prin porturile USB/ PS2 și utilizează întreruperi hardware (IRQ1) pentru a trimite scan-coduri către procesor la apăsarea/eliberarea tastelor.
- **Caracteristici:** Polling rate (frecvența de interogare), key travel/actuation (distanța de apăsare).
- **Categorii taste:** Taste alfanumerice, taste funcționale (F1-F12), taste de navigare.
- **Shortcut-uri:** Ctrl+C (copiere), Ctrl+V (lipire), Alt+Tab (schimbare aplicații).

3. Programare: Cifrele minime și maxime distincte

Soluție C++:

```
#include <iostream>
#include <fstream>
#include <vector>
using namespace std;

int nrAp(long long n, int c) {
    if (n == 0 && c == 0) return 1;
    int count = 0;
    long long temp = n;
    while (temp > 0) {
        if (temp % 10 == c) count++;
        temp /= 10;
    }
    return count;
}

int main() {
    long long n;
    if (!(cin >> n)) return 0;

    int freq[10];
    for (int i = 0; i < 10; ++i) freq[i] = nrAp(n, i);

    long long smallest = 0;
    int first_non_zero = -1;
    for (int i = 1; i <= 9; ++i) {
        if (freq[i] > 0) { first_non_zero = i; break; }
    }

    if (first_non_zero == -1) {
        smallest = 0;
    } else {
        smallest = first_non_zero;
        freq[first_non_zero]--;
        for (int i = 0; i < 10; ++i) {
            for (int j = 0; j < freq[i]; ++j) smallest = smallest * 10 + i;
        }
        freq[first_non_zero]++;
    }

    long long largest = 0;
    for (int i = 9; i >= 0; --i) {
        for (int j = 0; j < freq[i]; ++j) largest = largest * 10 + i;
    }
}
```

```

    }

    ofstream fout("def2019.out");
    if (smallest == largest) {
        fout << "nu exista\n";
    } else {
        fout << smallest << " " << largest << "\n";
    }
    fout.close();
    return 0;
}

```

4. Baze de date: Agenție de impresariat muzical

- **Entități:**

- SOLIST: id_solist (PK), nume, prenume, data_nasterii.
- ROL: id_rol (PK), nume_personaj, tip_voce, spectacol.
- DISTRIBUTIE: id_solist (FK), id_rol (FK), an_distributie, PK este (id_solist, id_rol, an_distributie).

- **SQL:**

```

SELECT DISTINCT s.prenume, s.nume
FROM SOLIST s
JOIN DISTRIBUTIE d ON s.id_solist = d.id_solist
WHERE d.an_distributie = YEAR(CURDATE());

```